AETURNUS

A

SYNOPSIS REPORT OF PROJECT

Aeturnus – Digital Continuity & Legacy Platform

For the course FRONT END WEB DEVELOPMENT (26CA205PCB)

In partial fulfillment of requirements for the degree of

MASTER OF COMPUTER APPLICATIONS (MCA)

SUBMITTED BY:

Name: Shubham Kumar

Roll No.: 2628MCA0117

August 2026



SUBMITTED TO:

Dr. Shashank Bhardwaj

&

Ms. Sakshi Goel

Department of Computer Applications

KIET Deemed To Be University,

Delhi-NCR, Ghaziabad-Meerut Road,

Ghaziabad, Uttar Pradesh

CONTENTS

1. Abstract
2. Motivation
3. Introduction
3.1 Objectives
3.2 Future Scope
3.3 Findings
4. Analysis Phase
4.1 Requirement Analysis
4.2 Data Flow Diagram (DFD) / Use Case Diagram
4.3 System Flow
5. Technology Stack (Web Development)
6. System Design & Methodology
7. Expected Outcomes (Website / Web App)
8. References

1. ABSTRACT

Aeturnus – Digital Continuity & Legacy Platform is a privacy-conscious web application concept designed to help individuals organize important digital information and define how selected information should remain accessible to trusted people when the user is temporarily or permanently unavailable. The project addresses a practical continuity problem created by the fragmentation of important information across financial services, insurance providers, email, cloud storage, documents, mobile devices and other digital channels.

Rather than functioning only as a password vault or conventional cloud drive, Aeturnus introduces a structured Digital Continuity Map. It connects important records with trusted people and predefined access policies. The proposed platform includes a Life Map / Asset Locker, Trusted People, Policy-Based Access, Continuity Score, Safe Check-In, Escalation Protocol, Audit Timeline and separate Emergency, Continuity and Legacy modes.

The Semester 1 academic implementation follows a frontend-first, backend-ready approach using HTML5, CSS3, JavaScript ES6+, React.js and Vite, with local/mock data for demonstration. The architecture is designed so that Semester 2 can introduce REST APIs, Node.js/Express.js, persistent database storage, authentication, scheduled server-side automation and audit logging without rebuilding the user interface.

The project follows security-by-design principles by avoiding real UPI PINs, ATM PINs, CVVs, banking passwords and private cryptographic keys in the academic prototype.

2. MOTIVATION

The motivation for selecting Aeturnus comes from the growing dependence on digital information in everyday life. Important records are no longer maintained only in physical files; they are distributed across online accounts, emails, cloud services, digital documents, financial and insurance portals, mobile devices and messaging platforms.

This fragmentation creates a continuity gap. During an emergency, long absence or a verified legacy situation, a trusted family member or nominee may know that important information exists but may not know where it is, what is relevant, who should receive it, or what level of access is appropriate. Giving someone unrestricted access to everything is also unnecessary from a privacy perspective.

Aeturnus was selected because it converts this real-world problem into a strong web-development project involving interface design, responsive layouts, JavaScript interactivity, state management, validation, React components and future backend architecture.

- Real-world relevance: digital information continuity is increasingly important for individuals and families.
- User importance: organized records and defined trusted access can reduce confusion during difficult situations.
- Web-development relevance: the problem naturally supports dashboards, forms, routing, responsive design and interactive workflows.
- Emerging technology alignment: the architecture can later incorporate APIs, authentication, automation and intelligent recommendations.
- Academic suitability: the project demonstrates HTML/CSS, JavaScript ES6+, React, UI/UX, validation, Git/GitHub and future full-stack integration.

3. INTRODUCTION

Aeturnus belongs to the domain of Web Development, Advanced Frontend Systems and Digital Information Management. It is proposed as a responsive web application for desktop, tablet and mobile devices. The primary users are individuals who want to organize important information and define controlled continuity arrangements for trusted contacts, nominees and executors.

3.1 Objectives

- Design a premium, responsive and accessible frontend that makes a sensitive subject feel calm, simple and trustworthy.
- Apply component-driven React architecture and reusable ES6 modules.
- Implement multi-step forms with robust client-side validation and clear error/success states.
- Build an interactive continuity engine using JavaScript state transitions and timing mechanisms such as setInterval/clearInterval.
- Create an API-ready data layer so local/mock data can later be replaced by REST APIs.
- Demonstrate secure-by-design thinking by avoiding raw banking passwords, UPI PINs, CVVs and private keys.
- Prepare a clear path for Semester 2 backend integration involving authentication, database persistence, scheduled jobs and audit logging.
- Demonstrate a complete user journey suitable for MSE-1, MSE-2, final presentation and viva.

3.2 Future Scope

- Secure document storage and stronger encryption architecture.
- Family/shared continuity plans with granular roles.
- Integration with email/document parsing for low-manual-entry onboarding.
- Time-bound emergency access and revocation workflows.
- Advanced continuity scoring and intelligent recommendations.
- Mobile application or PWA extension.
- Institutional or professional use cases for estate planners, legal executors and family offices.
- Interoperability with future digital-product, identity or verifiable-credential ecosystems.

3.3 Findings

- Digital information is increasingly distributed across multiple services and devices.
- Storage alone does not solve future access and continuity.
- Different records require different access levels and trusted recipients.
- Privacy and least-privilege access are important design requirements.
- A missed check-in should not be treated as definitive proof of death.
- Service abstractions allow the frontend to remain independent from the future backend.

4. ANALYSIS PHASE

The analysis phase identifies the functional and non-functional requirements of Aeturnus and converts the continuity problem into a structured workflow around users, records, trusted people, policies, continuity states and controlled access.

4.1 Requirement Analysis

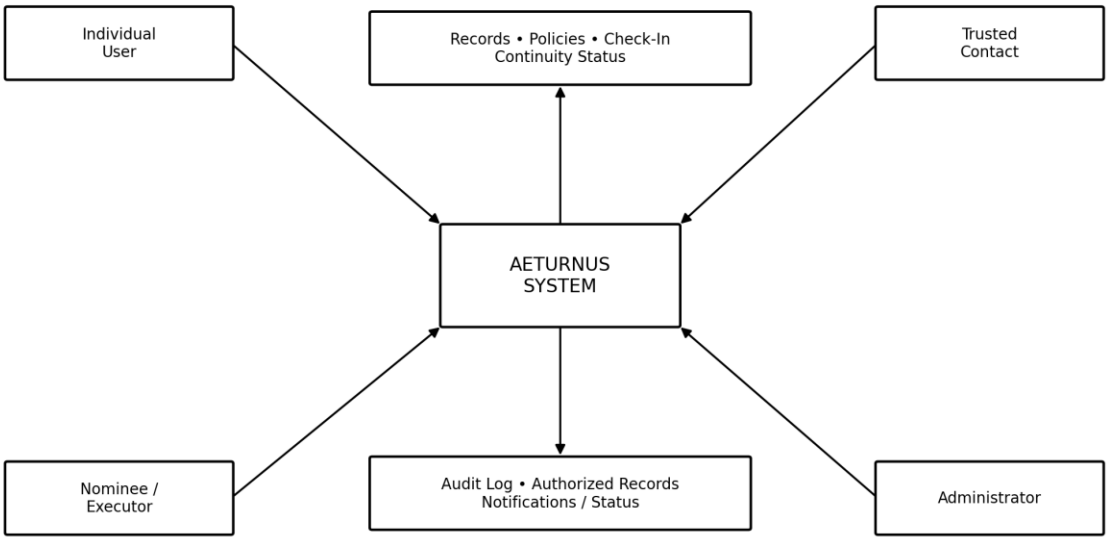
4.1.1 Functional Requirements

- User registration and mock login with validation.
- Profile and security settings management.
- Creation, viewing, updating and organization of important records.
- Categories such as Financial, Insurance, Documents, Property, Subscriptions and Emergency Information.
- Trusted people with roles such as Family Contact, Trusted Contact, Nominee and Executor.
- Policy-based mapping of selected records to selected people.
- Limited permissions rather than unrestricted access.
- Continuity dashboard showing status, next check-in, continuity score and attention items.
- Safe Check-In functionality.
- Simulated Active → Reminder → Warning → Grace Period → Verification → Controlled Release workflow.
- Trusted-contact/nominee portal displaying only policy-authorized records.
- Audit timeline for check-ins, policy changes, state changes and access events.

4.1.2 Non-Functional Requirements

- Security, privacy, usability, responsiveness, performance, scalability, reliability and maintainability.
- The application should provide clear error handling and should remain usable across desktop, tablet and mobile devices.

4.2 Data Flow Diagram (DFD) / Use Case Diagram



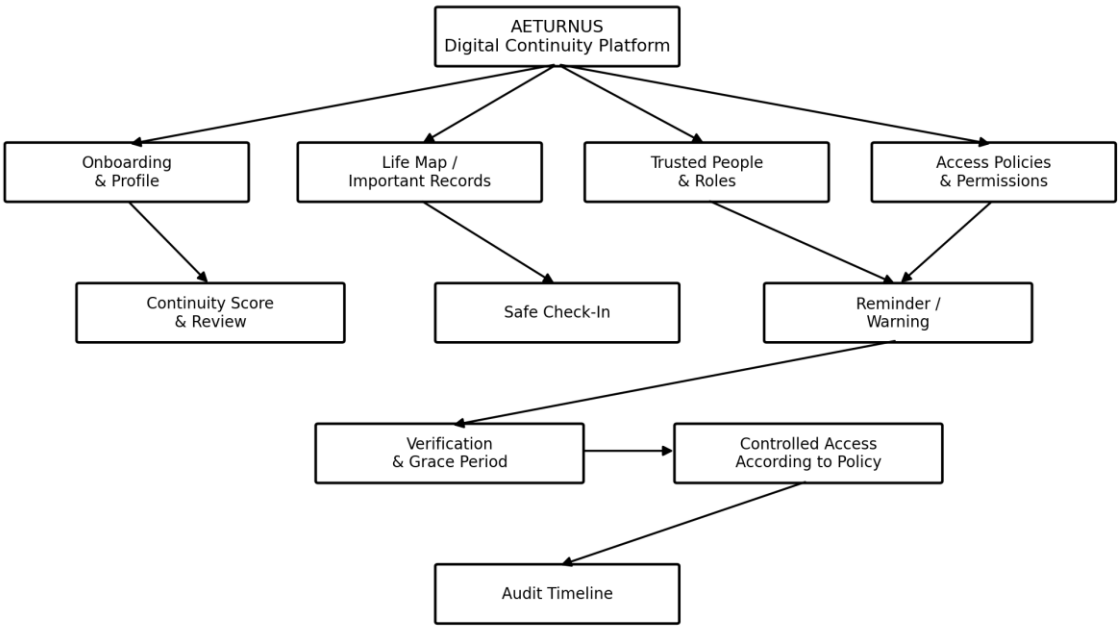
Conceptual system boundary and information flow:

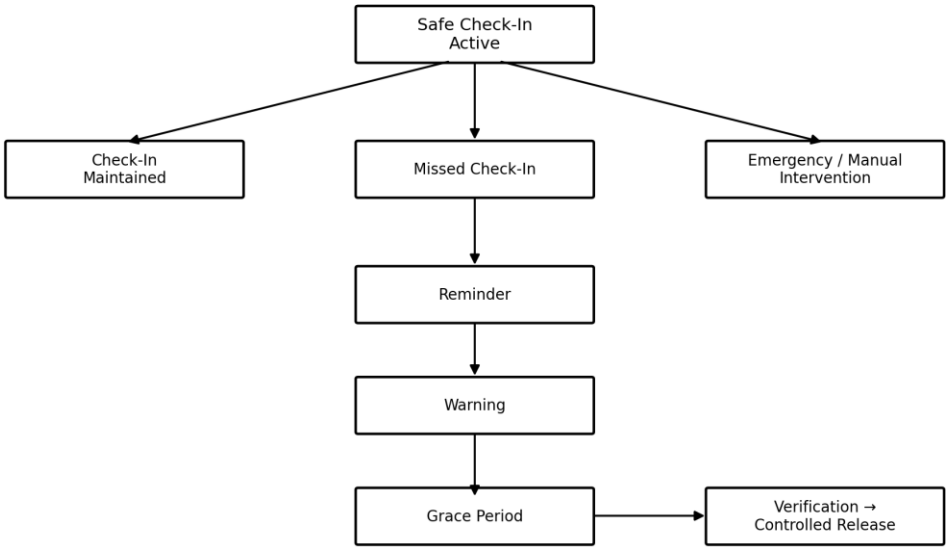
Actor / Store	Aeturnus Process	Main Flow
User	Authentication & Profile	Login / profile data ↔ system
User	Life Map / Asset Locker	Create / update / view important records
User	Trusted People	Add contacts and roles
User	Access Policy	Assign records and permissions
User	Continuity Engine	Check-in → reminder → warning → grace period → verification
Trusted Contact	Authorized Portal	Receive only policy-approved information
System Data Store	Audit Timeline	Store state and access history

Use case summary:

Actor	Major Use Cases
Individual User	Register, Login, Manage Profile, Manage Records, Add Trusted People, Define Policies, Check In, Review Continuity Status
Trusted Contact	Authenticate, View Authorized Information, Follow Assigned Access Policy
Nominee / Executor	Receive selected records according to predefined policy after an approved continuity condition
Administrator	Manage users, monitor system and maintain platform

4.3 System Flow





5. TECHNOLOGY STACK (WEB DEVELOPMENT)

The technology stack is selected to satisfy the Front End Web Development course requirements while keeping the architecture ready for future full-stack integration.

Layer / Area	Technology	Purpose
Structure	HTML5	Semantic structure and accessible web content
Styling	CSS3	Responsive layout, typography, spacing and visual design
Programming	JavaScript ES6+	Validation, state transitions, timers and client-side utilities
Frontend Framework	React.js	Component-based UI and reusable application modules
Build Tool	Vite	Development server and production build
Semester 1 Data	LocalStorage / Mock JSON / Mock API	Local demonstration and persistence
Future Backend	Node.js + Express.js	REST API layer and server-side processing
Future Database	MongoDB / PostgreSQL	Persistent application storage
Version Control	Git + GitHub	Source control and feature-based commits
UI/UX	Figma	Wireframes, components and user-flow visualization
API Testing	Postman	Future REST API request/response testing
Deployment	Vercel / Netlify	Live hosting

5.1 API-Ready Data Contract

Frontend Service	Future REST Endpoint	Purpose
getUser()	GET /api/user	Fetch user profile
getAssets()	GET /api/assets	Fetch important records
createAsset()	POST /api/assets	Create a new record
updatePolicy()	PUT /api/policies/:id	Update access rules
performCheckIn()	POST /api/check-ins	Record a secure check-in
getContinuityStatus()	GET /api/continuity/status	Return current continuity state
getAuditLog()	GET /api/audit-log	Display activity/state history

6. SYSTEM DESIGN & METHODOLOGY

Aeturnus follows a frontend-first architecture for the current academic scope. React components communicate through defined data models and service functions, creating a clean separation between the user interface and the data layer.

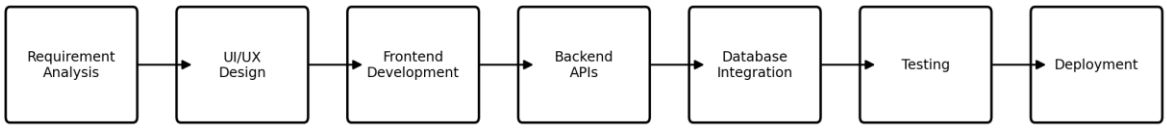
6.1 System Design

Layer	Main Responsibility
Presentation Layer	React components, HTML5, CSS3 and responsive UI
Application Logic Layer	JavaScript ES6+, validation, state management and continuity transitions
Service / Data Layer	Mock/local services in Semester 1; REST services in Semester 2
Backend Layer – Future	Node.js + Express.js APIs, authentication and authorization
Database Layer – Future	MongoDB or PostgreSQL persistent storage

6.2 Core Modules

- Authentication & Profile Module – mock signup/login, profile and security settings with validation.
- Life Map / Asset Locker – structured catalog of financial, insurance, document, property and responsibility metadata.
- Trusted People Module – add contacts and define their roles.
- Access Policy Module – map selected assets to selected people with limited permissions.
- Continuity Dashboard – display safety status, next check-in, continuity score and attention items.
- Continuity Engine – manage Active → Reminder → Warning → Grace Period → Verification → Controlled Release states.
- Nominee / Trusted Contact Portal – display only records authorized by the policy.
- Audit Timeline – show check-ins, policy updates, state changes and future access events.

6.3 Methodology



Phase	Planned Activity
1. Requirement Analysis	Identify the problem, users, requirements and constraints.
2. UI/UX Design	Create user flows, wireframes and responsive layouts in Figma.

3. Frontend Development	Build reusable React components using HTML5, CSS3, JavaScript ES6+ and Vite.
4. Backend API Planning	Define service functions and future REST endpoint contracts.
5. Database Integration	Introduce persistent storage in Semester 2 using MongoDB or PostgreSQL.
6. Testing	Perform form, component, workflow, responsive and failure-state testing.
7. Deployment	Deploy locally and/or to Vercel/Netlify.

6.4 Security & Privacy-by-Design

- Use dummy records and non-sensitive metadata in the academic prototype.
- Do not store real UPI PINs, ATM PINs, CVVs, banking passwords or private cryptographic keys.
- Require explicit consent before adding trusted people or assigning access policies.
- Release the minimum necessary information to each role.
- Use time-bound and revocable emergency access in future production versions.
- Maintain an audit trail for important security-sensitive actions.
- Keep emergency continuity separate from legal inheritance.
- Never treat a missed check-in alone as definitive proof of death.

6.5 Testing Strategy

- Form validation testing – required fields, formats, masking and boundary cases.
- Component testing – reusable components and state updates.
- Workflow testing – add record, assign policy, check-in and escalation states.
- Responsive testing – mobile, tablet and desktop breakpoints.
- Security-oriented UI testing – unauthorized route access and incorrect role visibility.
- Failure-state testing – invalid data, failed requests, expired sessions and missed check-in scenarios.
- Future API contract testing – request/response structures and error handling.

7. EXPECTED OUTCOMES (WEBSITE / WEB APP)

The project is expected to produce a responsive, multi-device web application prototype that demonstrates the complete continuity concept while remaining suitable for future backend integration.

- A professional responsive frontend for Aeturnus – Digital Continuity & Legacy Platform.
- A clear home/onboarding experience and structured navigation.
- Life Map / Asset Locker for organizing important information.
- Validated multi-step forms for records and trusted contacts.
- Trusted People and Access Policy interfaces.
- Continuity Dashboard showing status, check-in information and continuity score.
- Interactive continuity/check-in workflow with simulated reminders and state transitions.
- Controlled trusted-contact/nominee portal showing only authorized records.
- Audit Timeline for continuity and access events.
- Reusable React components and ES6 modules.

- Mock/local data layer with API-ready service abstraction.
- Responsive behavior across desktop, tablet and mobile devices.
- Git/GitHub-based project organization and feature-based development.
- A clear Semester 2 migration path for REST APIs, authentication, database storage, scheduled jobs and audit logging.

7.1 Planned Demonstration Journey

1. Create or open the user profile.
2. Add an important record to the Life Map.
3. Add a trusted person and define the role.
4. Assign a limited access policy to the selected record.
5. Review the continuity score and attention items.
6. Perform a safe check-in.
7. Demonstrate reminder/warning states using the simulated continuity engine.
8. Show the grace-period and verification concept.
9. Open the controlled-access view and demonstrate policy-limited information.
10. Review the audit timeline.

7.2 Limitations and Ethical Boundaries

Aeturnus is an academic prototype and does not claim to perform legal inheritance, banking transactions, probate, cryptocurrency transfer or automatic proof of death. Legal authority remains with applicable laws, institutions, nominees and executors. The prototype focuses on information continuity, controlled access and workflow automation. A production implementation would require legal review, threat modelling, professional security auditing and compliance assessment.

7.3 Semester 2 Expansion

- Node.js + Express.js REST API layer.
- MongoDB or PostgreSQL persistent storage.
- Authentication and authorization using secure patterns.
- Password hashing using bcrypt or an equivalent secure mechanism.
- Role-based and policy-based access control.
- Server-side scheduled jobs/Cron for check-in monitoring and notification workflows.
- Encrypted storage for sensitive metadata/documents where appropriate.
- Notification service for reminders, warnings and approved protocol events.
- Audit logging for policy changes, check-ins, verification and controlled access.
- Migration from mock/local service functions to production API services.

8. REFERENCES

11. React Documentation – official React documentation and learning resources.
12. MDN Web Docs – HTML, CSS, JavaScript and Web API documentation.
13. Vite Documentation – official Vite documentation.
14. Node.js Documentation – official Node.js documentation.
15. Express.js Documentation – official Express framework documentation.
16. MongoDB Documentation – official MongoDB documentation.
17. OWASP Foundation – web application security and secure development guidance.
18. Figma Documentation – UI/UX design and prototyping resources.
19. Sommerville, Ian. Software Engineering. Pearson.
20. Pressman, Roger S. and Maxim, Bruce R. Software Engineering: A Practitioner's Approach. McGraw-Hill.

APPENDIX A – PROJECT SNAPSHOT

Item	Aeturnus
Project Title	Aeturnus: A Digital Continuity & Legacy Platform
Domain	Web Development / Advanced Frontend Systems / Digital Information Management
Project Type	Responsive Web Application – Frontend-first, Backend-ready
Primary Users	Individuals, families, trusted contacts, nominees and executors
Current Academic Scope	Semester 1: HTML5, CSS3, JavaScript ES6+, React.js, UI/UX, Git/GitHub
Future Academic Scope	Semester 2: REST APIs, Node.js/Express.js, database, authentication and server-side automation
Core Idea	Digital Continuity Map connecting important records, trusted people and access policies